Agentic Orchestration Hackathon Brief

Overview

Build **autonomous agents** that follow agentic design patterns and orchestration architecture. Each agent should reason about problems, plan solutions, and execute tools autonomously while verifying results at multiple levels.

Theme: Agents with intelligent orchestration across diverse domains.

Project Scope

Participating Domains

Teams will build reference agents in **6 domains**:

1. **IT Infrastructure Automation** — Provision, configure, and monitor infrastructure
2. **Finance** — Analyze transactions, generate reports, detect anomalies
3. **Healthcare** — Manage patient data, schedule appointments, generate insights
4. **Personal Assistants** — Task management, scheduling, data organization
5. **Application Development Lifecycle** — CI/CD management, deployment automation, issue tracking
6. **Real Estate** — Property valuation, rental management, investment analysis

Each domain gets **one independent agent** with consistent architecture patterns.

Architecture Requirements

Agent Design

- **5-6 tools minimum** per agent
- **Independent agents** (no cross-agent dependencies; each is autonomous)
- **Loop-with-feedback orchestration** following agentic design patterns
- **Max 2-3 retries** per execution step before escalation
- **Optional:** LangGraph for state management (any orchestration framework acceptable as long as patterns are followed)

Agentic Pattern: Observe → Reason → Plan → Act → Verify → Iterate

Each agent must follow this cycle:

1. **Observe:** Understand the task and current state (read data sources)
2. **Reason:** Analyze the situation and decide which tool(s) to use
3. **Plan:** Decompose task into steps, create execution plan with TODOs
4. **Act:** Execute tools in sequence or parallel with error handling
5. **Verify:** Check if results meet the goal/specification
6. **Iterate:** If verification fails, replan and retry (up to max retries)

Tool Architecture

Required Tool Categories (6 minimum per agent)

Every agent must have at least these tool types:

Tool Category	Purpose	Example
Retrieval	Fetch data from sources (API, CSV, database)	Read VM inventory, fetch transactions
Processing	Transform, filter, aggregate data	Normalize data, compute statistics
Execution	Perform actions/state changes	Provision VM, deploy application
Verification	Validate results against specification	Check VM was created, audit logs
Analysis	Compute insights, generate recommendations	Anomaly detection, performance metrics
Reporting	Format and summarize output	Generate reports, structure responses

Tool Design Principles

- **Pure functions:** Tools have no side effects outside their scope; same inputs always produce same outputs
- **Immutable inputs:** Tools receive immutable data structures
- **Standardized returns:** All tools return `{status, data, metadata}` structure

- **Atomic operations:** Each tool performs one focused task
- **Comprehensive logging:** All executions logged with timestamps and inputs/outputs

---

Verification Strategy (3 Layers)

Layer 1: Tool-Level Verification

- Each tool validates its own output before returning
- Simple checks: API success codes, non-null responses, data schema validation
- Example: Provision-VM tool verifies VM actually exists after creation

Layer 2: Agent-Level Verification

- After executing a step, agent checks if result semantically meets the goal
- Determines if the output is sufficient to proceed or requires retries
- Example: Agent checks "does this VM configuration match the specification?"

Layer 3: End-to-End Verification

- Final deliverable is reconciled against original task specification
- Full state consistency check
- Example: Infrastructure state audit comparing desired vs. actual configuration

---

Coding Standards & Practices

Code Organization

- **Clear separation:** Tools in dedicated modules, orchestration logic isolated, state management separated
- **Consistent naming:** Tool classes follow `{Domain}{Action}Tool` pattern (e.g., `AWSProvisionVMTool`)
- **Type safety:** Use type hints (TypeScript/Python) for all function signatures

Error Handling & Recovery

- **Fail-safe defaults:** System continues gracefully when tools fail
- **Retry logic:** Exponential backoff with max 2-3 retries per step
- **Rollback capability:** Agents can undo previous steps if later steps fail (where applicable)
- **Comprehensive logging:** Every tool execution, decision point, and error logged with context

Testing Requirements

- **Unit tests:** Test each tool independently with mock data
- **Integration tests:** Test agent orchestration workflows end-to-end
- **Test scenarios:** Provide 3-5 realistic use cases per domain (documented in README)
- **Success criteria:** Agent executes end-to-end workflow + verification passes

Documentation Requirements

- **Tool docstrings:** Clear purpose, inputs, outputs, error conditions
- **Agent README:** Purpose, use cases, tool overview, data sources, example workflows, testing instructions
- **Architecture diagram:** Visual of orchestration flow (optional but recommended)
- **Data schema documentation:** For each data source used

Data Management & Local-Only Execution

Critical Constraint: Local Development, No Subscriptions

All agents must be runnable on a local development machine with NO external subscriptions, cloud credentials, or paid services.

- No AWS, GCP, Azure credentials required
- No paid API subscriptions
- No external cloud dependencies
- Local data (CSV, SQLite, mock data)
- Free, open-source APIs
- Local infrastructure (Docker, VirtualBox)

Data Sources

- **Open-source only:** Use public APIs and publicly available datasets
- **Flatfiles:** CSV, TSV, JSON files for structured data
- **Kaggle datasets:** For realistic data in Healthcare and Finance domains
- **No data generation scripts:** Provide sample data or document download links
- **Simulation-based:** For agents that normally interact with external systems (Finance, Personal Assistant, Healthcare)
- **Real local infrastructure:** For IT Infrastructure (Docker Compose, VirtualBox, local containers)

Data Organization

```
/data/
  ├── it_infrastructure/
  │   ├── vm_inventory.csv
  │   ├── network_config.csv
  │   └── README.md
  ├── finance/
  │   ├── transactions.csv
  │   ├── accounts.csv
  │   └── README.md
  ├── healthcare/
  │   ├── patients.csv
  │   ├── appointments.csv
  │   └── README.md
  ├── personal_assistant/
  │   ├── tasks.csv
  │   ├── calendar.csv
  │   └── README.md
  └── app_dev_lifecycle/
      ├── repositories.csv
      ├── deployments.csv
      └── README.md
```

Data Source Examples

- **IT Infrastructure:** Kaggle datasets, open cloud APIs (AWS, GCP, Azure free tier)
- **Finance:** Yahoo Finance API, data.gov datasets, public transaction samples
- **Healthcare:** MIMIC dataset, synthetic patient data from healthcare open-source projects
- **Personal Assistant:** Task/calendar data in standard formats
- **App Dev:** GitHub API, GitLab open datasets, public CI/CD logs

---

Output Standardization (Loose)

Recommended Output Schema

While standardization is loose, agents should follow this spirit:

```
{
  "success": true/false,
  "result": "<domain-specific result>",
  "verification_status": "passed|failed|partial",
  "steps_executed": ["step1", "step2", "step3"],
  "tools_used": ["tool1", "tool2"],
```

```
  "execution_time_seconds": 2.34,
  "errors": ["optional error messages"],
  "metadata": {
    "retries": 0,
    "domain": "it_infrastructure"
  }
}
```

Flexibility: Teams can extend or adapt this schema for domain-specific needs, but core fields should be present.

Success Criteria

Minimum Requirements

- Agent executes end-to-end workflow without manual intervention
- All 3 verification layers pass (tool, agent, end-to-end)
- Agent follows Observe → Reason → Plan → Act → Verify → Iterate cycle
- 5-6 tools implemented with correct categories
- Comprehensive logging at all decision points
- Unit tests for tools + integration test for full workflow
- Documentation (README, docstrings, architecture)
- Data sources provided or documented

Evaluation Criteria

1. **Architecture (25%)** — Does agent follow agentic patterns? Is orchestration clear?
2. **Tool Implementation (25%)** — Are tools well-designed, tested, and documented?
3. **Verification (20%)** — Are results verified at all 3 layers? Is verification robust?
4. **Code Quality (15%)** — Is code clean, well-organized, type-safe, well-documented?
5. **Demo (15%)** — Can you show the agent working on realistic scenarios? Does it handle errors gracefully?

Technology Stack (Recommended)

- **Languages:** Python or TypeScript/JavaScript
- **Agent Frameworks:** LangChain (recommended), optional LangGraph for state management
- **LLM:** Claude or OpenAI (with proper API key management)
- **Data:** CSV/JSON files, no external databases required

- **Testing:** pytest (Python) or Jest (TypeScript)
- **Logging:** Python logging or Winston/Bunyan (TypeScript)

Team Deliverables

Submission Structure

```
agent-{domain}/
├── README.md (overview, tools, data sources, testing)
├── src/
│   ├── agent.py (or .ts) — orchestration logic
│   ├── tools/
│   │   ├── retrieval_tool.py
│   │   ├── processing_tool.py
│   │   ├── execution_tool.py
│   │   ├── verification_tool.py
│   │   ├── analysis_tool.py
│   │   └── reporting_tool.py
│   └── state.py (state management)
├── tests/
│   ├── test_tools.py — unit tests
│   └── test_agent.py — integration tests
├── data/
│   ├── sample_input.csv
│   └── expected_output.json
└── docs/
    ├── ARCHITECTURE.md
    └── DATA_SCHEMA.md
```

What to Demo

1. **Task execution:** Show agent solving a realistic scenario
2. **Planning:** Show how agent decomposes the task
3. **Tool execution:** Show individual tools being called with logging
4. **Verification:** Show verification passing at all 3 levels
5. **Error handling:** Show graceful failure and retry (if applicable)

FAQ & Constraints

Can we use external APIs?

Yes, but only open-source/free APIs (no paid services, no proprietary data).

Can we use databases?

Prefer flatfiles (CSV/TSV/JSON). If you need structured data access, SQLite with provided sample data is acceptable.

Can we use LangGraph?

Yes, but it's optional. Any orchestration framework works as long as you follow the state machine pattern.

What if a tool fails?

Implement graceful error handling: log the error, retry up to 2-3 times with backoff, then escalate or skip the step (agent decides).

How many agents do we build?

Build ONE agent (pick one domain). You don't need to build all 5 — this brief covers all 5 domains so multiple teams can each build one.

What if verification fails?

Agent replans and retries (up to max retries). If still failing, return failure status with clear error messages.

Timeline & Milestones

- **Phase 1:** Design agent architecture, define tools, set up data
- **Phase 2:** Implement tools and verification logic
- **Phase 3:** Build orchestration (Observe → Plan → Act → Verify loop)
- **Phase 4:** Testing, error handling, logging
- **Phase 5:** Documentation, demo prep

Getting Started

1. **Pick a domain** (IT Infrastructure, Finance, Healthcare, Personal Assistant, or App Dev Lifecycle)
2. **Define your 5-6 tools** using the required categories
3. **Set up data sources** (CSV files or API integrations)
4. **Implement orchestration loop** following Observe → Reason → Plan → Act → Verify → Iterate

5. **Build verification** at all 3 layers
6. **Write tests** and documentation
7. **Demo** your working agent

Resources & References

- **LangChain Documentation:** https://python.langchain.com/
- **LangGraph:** https://langchain-ai.github.io/langgraph/
- **Claude API:** https://anthropic.com/docs
- **Open Datasets:** Kaggle, data.gov, GitHub
- **Example Agent Patterns:** ReAct, Chain-of-Thought reasoning

---

Contact & Support

For questions about the brief or clarifications on requirements, reach out to the hackathon organizers.

Good luck! Build intelligent, autonomous agents!